Course Name: **ALGORITHMS & AI LABORATORY**

Course Code: **MCSL106**

1. Implement a simple linear regression algorithm to predict a continuous target variable based on a given dataset.

Algorithm for Linear Regression

Input:

- A dataset (`df`) with features (`X`) and target (`Y`).

Steps:

1. **Load the Dataset**
   - Read the dataset from a CSV file (`linear_dataset.csv`).
2. **Preprocess the Data**
   - Extract the independent variable (`X`) and dependent variable (`Y`).
3. **Train the Linear Regression Model**
   - Initialize the `LinearRegression` model.
   - Fit the model using the training data (`X, Y`).
   - Compute the predicted values (`y_pred`) for the given `X`.
4. **Plot the Results**
   - Scatter plot the actual data points (`X`, `Y`).
   - Plot the **best-fit regression line** using predicted values (`y_pred`).
5. **Evaluate the Model**
   - Calculate **Mean Squared Error (MSE)** to measure prediction error.
   - Compute **R-squared Score (R^2)** to determine model accuracy.
   - Extract and print the **intercept** and **coefficient(s)** of the model.
6. **Output the Results**
   - Display MSE, R^2 score, intercept, and coefficient(s).
   - Show the regression plot.

Output:

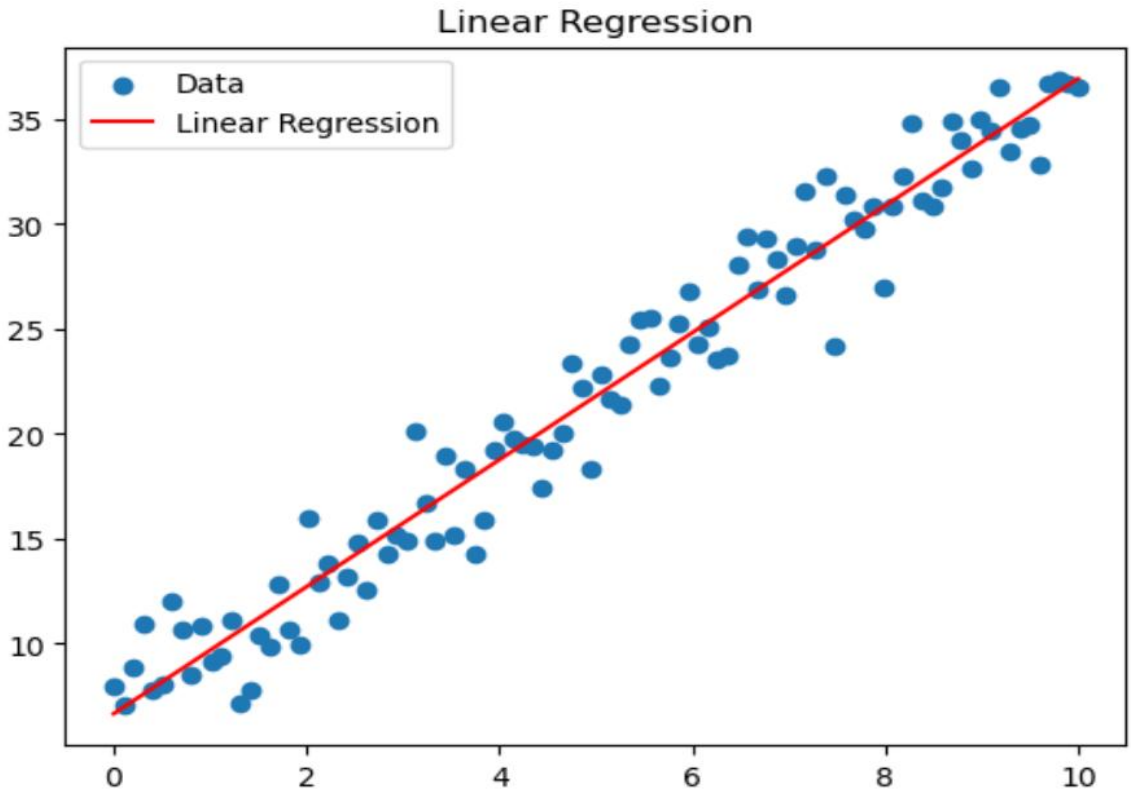
- **Numerical Results:** MSE, R^2 score, intercept, and coefficient.
- **Graphical Output:** Scatter plot with a red regression line.

CODE:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
from scipy.spatial.distance import cdist
from sklearn.metrics import mean_squared_error, r2_score

# Load datasets
df_linear = pd.read_csv("linear_dataset.csv")
df_lwr = pd.read_csv("lwr_dataset.csv")

# Linear Regression
def linear_regression(df):
    X, y = df[['X']], df['Y']
    model = LinearRegression()
    model.fit(X, y)
    y_pred = model.predict(X)

    plt.scatter(X, y, label='Data')
    plt.plot(X, y_pred, color='red', label='Linear Regression')
    plt.legend()
    plt.title("Linear Regression")
    plt.show()

    # Evaluating the model
    mse = mean_squared_error(y, y_pred)
    r2 = r2_score(y, y_pred)

    # Printing results
    print(f'Mean Squared Error: {mse}')
    print(f'R-squared Score: {r2}')
    print(f'Intercept: {model.intercept_}')
    print(f'Coefficient: {model.coef_[0]}')

linear_regression(df_linear)
```

Output:

```
Mean Squared Error: 3.259618853992314
R-squared Score: 0.95985119104905
Intercept: 6.654374291845247
Coefficient: 3.0275865346733135
```



2. Develop a program to implement a Support Vector Machine for binary classification. Use a sample dataset and visualize the decision boundary.

Algorithm for SVM Classification

Input:

- Iris dataset with features (Sepal Length & Width)
- Target labels (Setosa & Versicolor)

Steps:

1. **Load the Dataset**
   - Extract the first two features (Sepal length, Sepal width).
   - Select only two classes (Setosa & Versicolor) for binary classification.
2. **Preprocess Data**
   - Standardize the features using `StandardScaler`.
   - Split the dataset into **training (80%)** and **testing (20%)** sets.
3. **Train the SVM Model**
   - Use a **linear kernel** in the `SVC` classifier.
   - Train the model using `fit(X_train, y_train)`.
4. **Make Predictions**
   - Predict the test set using `predict(X_test)`.
   - Evaluate the model using **accuracy score**.
5. **Plot Decision Boundary**
   - Generate a mesh grid of values covering the feature space.
   - Predict labels for each grid point and plot the decision regions.
   - Scatter plot the data points with actual labels.

Output:

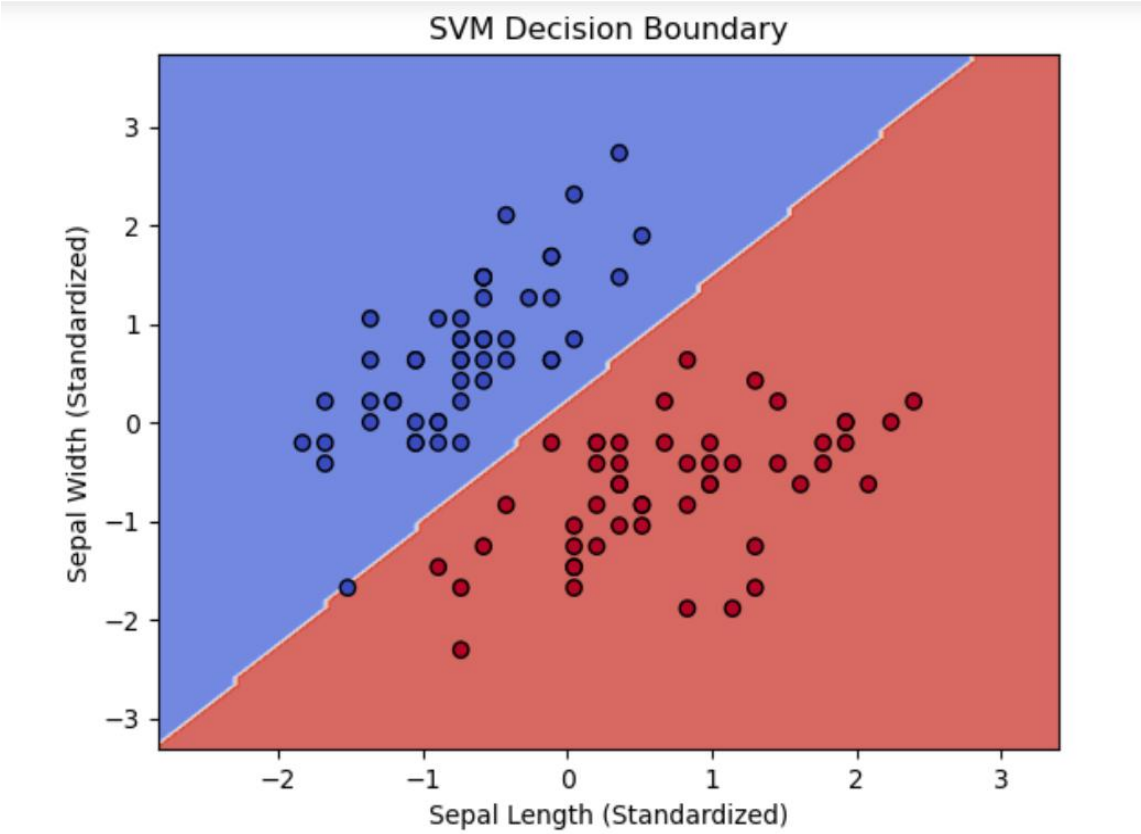
- **Model Accuracy** (e.g., ~1.00 for a well-separated dataset)
- **Decision Boundary Visualization** (Separation between Setosa & Versicolor)

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

# Load dataset and select two classes (Setosa and Versicolor)
iris = load_iris()
X = iris.data[iris.target != 2, :2]  # Use first two features (Sepal length & width)
y = iris.target[iris.target != 2]  # Use Setosa and Versicolor only

# Standardize features
X = StandardScaler().fit_transform(X)
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train SVM model
model = SVC(kernel='linear')
model.fit(X_train, y_train)

# Predict and evaluate
y_pred = model.predict(X_test)
print(f"Model Accuracy: {accuracy_score(y_test, y_pred):.2f}")
# Plot decision boundary
def plot_decision_boundary(X, y, model):
    xx, yy = np.meshgrid(np.linspace(X[:, 0].min()-1, X[:, 0].max()+1, 100),
                         np.linspace(X[:, 1].min()-1, X[:, 1].max()+1, 100))
    Z = model.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
    plt.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.coolwarm)
    plt.scatter(X[:, 0], X[:, 1], c=y, edgecolors='k', cmap=plt.cm.coolwarm)
    plt.xlabel('Sepal Length (Standardized)')
    plt.ylabel('Sepal Width (Standardized)')
    plt.title('SVM Decision Boundary')
    plt.show()
plot_decision_boundary(X, y, model)
```

Output:

Model Accuracy: 1.00



3. **Develop a simple case-based reasoning system that stores instances of past cases. Implement a retrieval method to find the most similar cases and make predictions based on them.**

Algorithm:

1. Define a class `Case` with attributes:
- `features` (list of numerical values)
- `target` (associated outcome)

2. Define a class `CBRSystem` with:
- `cases` (an empty list to store past cases)

3. Implement `add_case(case)`:
- Append `case` to `cases` list

4. Implement `retrieve_best_match(query_features)`:
- Initialize `best_case` as the first stored case
- Initialize `min_distance` as a large value
- For each `case` in `cases`:
  - Compute `distance = sum(abs(a - b) for a, b in zip(case.features, query_features))`
  - If `distance` is smaller than `min_distance`:
    - Update `min_distance`
    - Update `best_case`
- Return `best_case.target`

5. For a given query:
- Call `retrieve_best_match(query_features)`
- Output the predicted target

Code:

```
class Case:
    def __init__(self, features, target):
        self.features = features
        self.target = target

class CBRSystem:
    def __init__(self):
        self.cases = []

    def add_case(self, case):
        self.cases.append(case)
```

```
    def retrieve_best_match(self, query_features):
        best_case = min(self.cases, key=lambda case: sum(abs(a - b) for a, b in zip(case.features,
query_features)))
        return best_case.target

# Example usage
cbr = CBRSystem()
cbr.add_case(Case([1, 2, 3], "A"))
cbr.add_case(Case([4,5, 6], "B"))
cbr.add_case(Case([7, 8, 9], "C"))

query = [3, 4, 5]
prediction = cbr.retrieve_best_match(query)
print(f"Prediction for {query}: {prediction}")
```

Output: `Prediction for [3, 4, 5]: B`

4. Write a program to demonstrate the ID3 decision tree algorithm using an appropriate dataset for classification.

Algorithm: ID3 Decision Tree

Input:

- A dataset with features X and target labels y

Steps:

1. **Load the Dataset**
   - Use the **Iris dataset** (4 features, 3 classes).
2. **Preprocess Data**
   - Split into **training (80%)** and **testing (20%)** sets.
3. **Build the Decision Tree**
   - Train an **ID3 Decision Tree** (`criterion='entropy'`) on the training data.
4. **Make Predictions**
   - Use the trained model to classify the test data.
5. **Evaluate the Model**
   - Calculate the **accuracy** using `accuracy_score(y_test, y_pred)`.
6. **Visualize the Decision Tree**
   - Plot the tree structure using `plot_tree()`.

Output:

- **Model Accuracy** (e.g., ~1.00 for the Iris dataset)
- **Decision Tree Visualization** showing feature splits

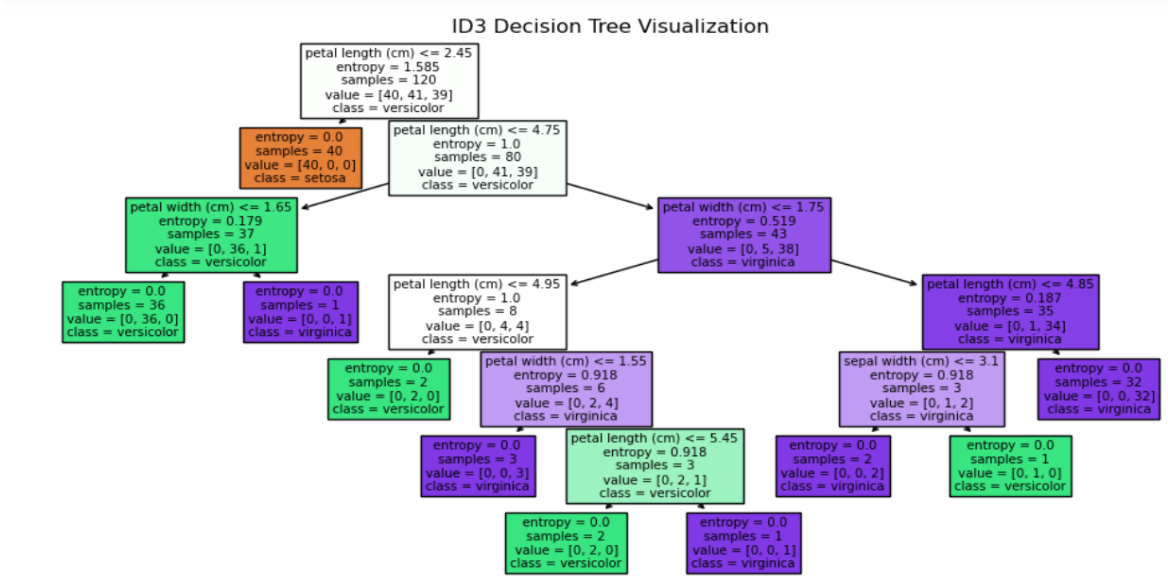
Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load the Iris dataset
iris = load_iris()
X = iris.data  # Features
y = iris.target  # Target labels
```

```
# Split dataset into training and testing sets (80% training, 20% testing)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train ID3 Decision Tree Classifier (criterion='entropy' for ID3)
id3_tree = DecisionTreeClassifier(criterion='entropy')
id3_tree.fit(X_train, y_train)

# Make predictions
y_pred = id3_tree.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {accuracy:.2f}")

# Visualize the Decision Tree
plt.figure(figsize=(12, 6))
plot_tree(id3_tree, feature_names=iris.feature_names, class_names=iris.target_names,
filled=True)
plt.title("ID3 Decision Tree Visualization")
plt.show()
```

Output:

Model Accuracy: 1.00


ID3 Decision Tree Visualization
petal length (cm) <= 2.45
entropy = 1.585
samples = 120
value = [40, 41, 39]
class = versicolor
entropy = 0.0
samples = 40
value = [40, 0, 0]
class = setosa
petal length (cm) <= 4.75
entropy = 1.0
samples = 80
value = [0, 41, 39]
class = versicolor
petal width (cm) <= 1.65
entropy = 0.179
samples = 37
value = [0, 36, 1]
class = versicolor
petal width (cm) <= 1.75
entropy = 0.519
samples = 43
value = [0, 5, 38]
class = virginica
entropy = 0.0
samples = 36
value = [0, 36, 0]
class = versicolor
entropy = 0.0
samples = 1
value = [0, 0, 1]
class = virginica
petal length (cm) <= 4.95
entropy = 1.0
samples = 8
value = [0, 4, 4]
class = versicolor
petal length (cm) <= 4.85
entropy = 0.187
samples = 35
value = [0, 1, 34]
class = virginica
entropy = 0.0
samples = 2
value = [0, 2, 0]
class = versicolor
petal width (cm) <= 1.55
entropy = 0.918
samples = 6
value = [0, 2, 4]
class = virginica
sepal width (cm) <= 3.1
entropy = 0.918
samples = 3
value = [0, 1, 2]
class = virginica
entropy = 0.0
samples = 32
value = [0, 0, 32]
class = virginica
entropy = 0.0
samples = 3
value = [0, 0, 3]
class = virginica
petal length (cm) <= 5.45
entropy = 0.918
samples = 3
value = [0, 2, 1]
class = versicolor
entropy = 0.0
samples = 2
value = [0, 0, 2]
class = virginica
entropy = 0.0
samples = 1
value = [0, 1, 0]
class = versicolor
entropy = 0.0
samples = 2
value = [0, 2, 0]
class = versicolor
entropy = 0.0
samples = 1
value = [0, 0, 1]
class = virginica

5. Build an Artificial Neural Network by implementing the Backpropagation algorithm and test it with suitable datasets.

Algorithm for Artificial Neural Network Using Backpropagation:

Step 1: Initialize the Neural Network

1. Define the number of input neurons, hidden layer neurons, and output neurons.
2. Initialize weights and biases randomly (typically small values).
3. Set the learning rate (α).

Step 2: Forward Propagation

1. Compute the input for the hidden layer: Hinput=XW1+B1H_{\text{input}} = X W_1 + B_1Hinput=XW1+B1
2. Apply the activation function (sigmoid): Houtput=σ(Hinput)H_{\text{output}} = \sigma(H_{\text{input}})Houtput=σ(Hinput)
3. Compute the input for the output layer: Oinput=HoutputW2+B2O_{\text{input}} = H_{\text{output}} W_2 + B_2Oinput=HoutputW2+B2
4. Apply the activation function: Ooutput=σ(Oinput)O_{\text{output}} = \sigma(O_{\text{input}})Ooutput=σ(Oinput)

Step 3: Compute Error

1. Calculate the error (difference between actual and predicted values): E=y−OoutputE = y - O_{\text{output}}E=y−Ooutput

Step 4: Backpropagation

1. Compute the gradient of the output layer: Oδ=E×σ′(Ooutput)O_{\delta} = E \times \sigma'(O_{\text{output}})Oδ=E×σ′(Ooutput)
2. Compute the gradient of the hidden layer: Hδ=OδW2T×σ′(Houtput)H_{\delta} = O_{\delta} W_2^T \times \sigma'(H_{\text{output}})Hδ=OδW2T×σ′(Houtput)

Step 5: Update Weights and Biases

1. Update the output layer weights and biases:

   W2=W2+HoutputTOδ×αW_2 = W_2 + H_{\text{output}}^T O_{\delta} \times \alphaW2 =W2+HoutputTOδ×α B2=B2+∑Oδ×αB_2 = B_2 + \sum O_{\delta} \times \alphaB2=B2 +∑Oδ×α

2. Update the hidden layer weights and biases: W1=W1+XTHδ×αW_1 = W_1 + X^T H_{\delta} \times \alphaW1=W1+XTHδ×α B1=B1+∑Hδ×αB_1 = B_1 + \sum H_{\delta} \times \alphaB1=B1+∑Hδ×α

Step 6: Repeat Training for Multiple Epochs

1. Repeat **Steps 2 to 5** for a set number of epochs or until the error is minimized.

Step 7: Make Predictions

1. For new inputs, perform **forward propagation** to get the output.

Code:

```
import numpy as np

# Sigmoid activation function and its derivative
def sigmoid(x):
  return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
  return x * (1 - x)

# XOR Neural Network with Hidden Layer
class XORNeuralNetwork:
  def __init__(self):
    np.random.seed(1)
# Weights from input layer to hidden layer (2 neurons)
    self.weights1 = np.random.rand(2, 2)
# Bias for hidden layer
    self.bias1 = np.random.rand(1, 2)
    self.weights2 = np.random.rand(2, 1)  # Weights from hidden layer to output layer (1 neuron)
    self.bias2 = np.random.rand(1, 1)  # Bias for output layer

  def forward(self, X):
    """Forward pass to compute predictions"""
    self.hidden_input = np.dot(X, self.weights1) + self.bias1
    self.hidden_output = sigmoid(self.hidden_input)
```

```
        self.final_input = np.dot(self.hidden_output, self.weights2) + self.bias2
        self.output = sigmoid(self.final_input)
        return self.output

    def backward(self, X, y, lr=0.5):
        """Backward propagation to adjust weights"""
        output_error = y - self.output  # Error in output
        output_delta = output_error * sigmoid_derivative(self.output)

        hidden_error = output_delta.dot(self.weights2.T)
        hidden_delta = hidden_error * sigmoid_derivative(self.hidden_output)

        # Update weights and biases
        self.weights2 += self.hidden_output.T.dot(output_delta) * lr
        self.bias2 += np.sum(output_delta, axis=0, keepdims=True) * lr

        self.weights1 += X.T.dot(hidden_delta) * lr
        self.bias1 += np.sum(hidden_delta, axis=0, keepdims=True) * lr

    def train(self, X, y, epochs=10000, lr=0.5):
        """Train the network"""
        for epoch in range(epochs):
            self.forward(X)
            self.backward(X, y, lr)

            # Print loss every 1000 epochs
            if epoch % 1000 == 0:
                loss = np.mean(np.abs(y - self.output))
                print(f"Epoch {epoch}, Loss: {loss:.5f}")

    def predict(self, X):
        """Predict output for given input"""
        return self.forward(X)
# XOR Dataset
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])  # Input data
y = np.array([[0], [1], [1], [0]])  # Expected output
# Train the network
nn = XORNeuralNetwork()
nn.train(X, y)
```

```
# Test predictions
print("\nFinal Predictions:")
for i, x in enumerate(X):
    prediction = nn.predict(x.reshape(1, -1))  # Reshaping input for compatibility
    rounded_prediction = np.round(prediction)[0][0]  # Rounding for binary classification
    print(f"Input: {x}, Raw Output: {prediction[0][0]:.5f}, Rounded Output:
{int(rounded_prediction)}")
```

Output:

```
Epoch 0, Loss: 0.49970
Epoch 1000, Loss: 0.32766
Epoch 2000, Loss: 0.06150
Epoch 3000, Loss: 0.04058
Epoch 4000, Loss: 0.03211
Epoch 5000, Loss: 0.02730
Epoch 6000, Loss: 0.02410
Epoch 7000, Loss: 0.02180
Epoch 8000, Loss: 0.02003
Epoch 9000, Loss: 0.01863

Final Predictions:
Input: [0 0], Raw Output: 0.01931, Rounded Output: 0
Input: [0 1], Raw Output: 0.98332, Rounded Output: 1
Input: [1 0], Raw Output: 0.98335, Rounded Output: 1
Input: [1 1], Raw Output: 0.01726, Rounded Output: 0
```

6. Implement a KNN algorithm for regression tasks instead of classification. Use a small dataset, and predict continuous values based on the average of the nearest neighbors.

Pseudocode for KNN Regression

1. Load dataset (X: features, y: continuous target values)
2. Normalize or standardize features (if necessary)
3. Split dataset into training and testing sets
4. Define K (number of nearest neighbors)
5. Store training data (X_train, y_train)

6. For each new test point X_test:
   a. Compute the Euclidean distance between X_test and all training points
   b. Sort distances in ascending order
   c. Select the k nearest neighbors
   d. Retrieve their target values (y_train)
   e. Predict the output as the average of selected y values

7. Evaluate the model using MSE or R^2 score
8. Visualize results (actual vs. predicted)

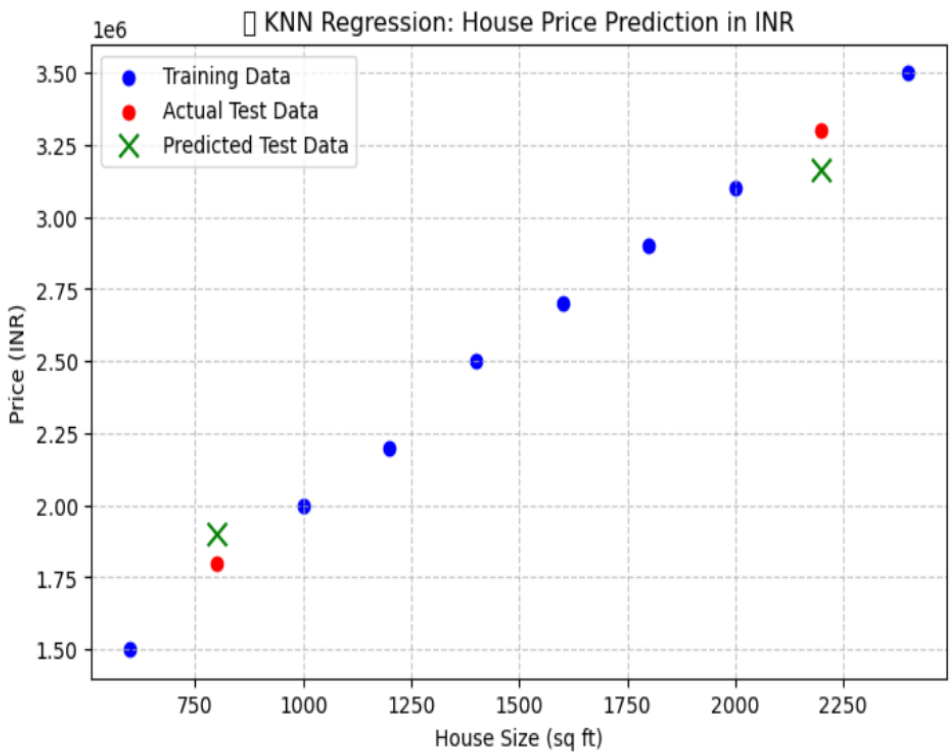
Code:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

class KNNRegressor:
    def __init__(self, k=3):
        self.k = k

    def fit(self, X, y):
        self.X_train = X
        self.y_train = y

    def predict(self, X_test):
        predictions = []
```

```
    for x in X_test:
        distances = np.linalg.norm(self.X_train - x, axis=1)  # Compute Euclidean distance
        nearest_indices = np.argsort(distances)[:self.k]  # Get k nearest neighbors
        nearest_values = self.y_train[nearest_indices]  # Fetch their corresponding y values
        predictions.append(np.mean(nearest_values))  # Take the average
    return np.array(predictions)

# Small Dataset: House Prices vs. Size (in square feet)
X = np.array([[600], [800], [1000], [1200], [1400], [1600], [1800], [2000], [2200], [2400]])
y = np.array([15, 18, 20, 22, 25, 27, 29, 31, 33, 35]) * 1e5  # House Prices in INR (Lakhs)

# Splitting Data into Training and Testing Sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Scaling (Important for distance-based models)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)


# Train the KNN Regressor Model
knn = KNNRegressor(k=3)
knn.fit(X_train_scaled, y_train)

# Predict on Test Data
y_pred = knn.predict(X_test_scaled)

# Print Predictions in INR
print("\nPredicted vs Actual House Prices (INR):\n")
for i in range(len(X_test)):
    print(f"Size: {X_test[i][0]} sq ft -> Predicted: ₹{y_pred[i]:,.2f}, Actual: ₹{y_test[i]:,.2f}")

# Visualization
plt.scatter(X_train, y_train, color="blue", label="Training Data")
plt.scatter(X_test, y_test, color="red", label="Actual Test Data")
plt.scatter(X_test, y_pred, color="green", label="Predicted Test Data", marker="x")
plt.xlabel("House Size (sq ft)")
plt.ylabel("Price (INR)")
plt.title(" KNN Regression: House Price Prediction in INR")
plt.legend()
plt.show()
```

Output:

```
Predicted vs Actual House Prices (INR):

Size: 2200 sq ft -> Predicted: ₹3,166,666.67, Actual: ₹3,300,000.00
Size: 800 sq ft -> Predicted: ₹1,900,000.00, Actual: ₹1,800,000.00
```



---

7. Create a program that calculates different distance metrics (Euclidean and Manhattan) between two points in a dataset. Allow the user to input two points and display the calculated distances.

Algorithm:

1. Load the house dataset with features (Size, Bedrooms, Price).
2. Display the dataset to the user.
3. Accept user input: two House IDs.
4. Convert House IDs to row indices in the dataset.
5. Extract feature vectors for the selected houses.
6. Calculate Euclidean Distance:

Use the formula:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$$

   a. Compute squared differences for each feature.
   b. Sum them and take the square root.

7. Calculate Manhattan Distance:

Use the formula:

$$d = |x_2 - x_1| + |y_2 - y_1| + |z_2 - z_1|$$

   a. Compute absolute differences for each feature.
   b. Sum them.

8. Display the Euclidean and Manhattan distances.

Code:

```
import numpy as np
import pandas as pd

#  Sample House Dataset
data = {
   "House ID": [1, 2, 3, 4, 5],
   "Size (sq ft)": [1000, 1200, 1500, 1800, 2000],
   "Bedrooms": [2, 3, 3, 4, 4],
   "Price (INR Lakhs)": [50, 65, 80, 95, 110]
}
```

```
#  Convert to DataFrame
df = pd.DataFrame(data)
print(" House Dataset:\n", df)

# Euclidean Distance Function
def euclidean_distance(p1, p2):
    return np.sqrt(np.sum((p1 - p2) ** 2))

#  Manhattan Distance Function
def manhattan_distance(p1, p2):
    return np.sum(np.abs(p1 - p2))

# Get user input (house indices)
house1 = int(input("\nEnter House ID for first house: ")) - 1
house2 = int(input("Enter House ID for second house: ")) - 1

# Extract feature vectors (Size, Bedrooms, Price)
point1 = df.iloc[house1, 1:].values
point2 = df.iloc[house2, 1:].values

# Compute distances
euclidean = euclidean_distance(point1, point2)
manhattan = manhattan_distance(point1, point2)

# Display results
print("\n Distance Metrics Between Houses:")
print(f" Euclidean Distance: {euclidean:.4f}")
print(f"Manhattan Distance: {manhattan:.4f}")
```

Output:

```
Enter House ID for first house: 1
Enter House ID for second house: 3
```

House Dataset:

House ID	Size (sq ft)	Bedrooms	Price (INR Lakhs)
0	1	1000	2	50
1	2	1200	3	65
2	3	1500	3	80
3	4	1800	4	95
4	5	2000	4	110

Distance Metrics Between Houses:

Euclidean Distance: 505.3785

Manhattan Distance: 580.0000

8. Implement the k-Nearest Neighbor algorithm to classify the Iris dataset, printing both correct and incorrect predictions.

Python implementation of the **k-Nearest Neighbors (KNN) algorithm** for classifying the **Iris dataset**. The program will:

1. **Load the Iris dataset**.
2. **Split the data** into training and testing sets.
3. **Implement KNN classification** using `k=5`.
4. **Print correct and incorrect predictions**.

Code:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

# Load the Iris dataset
iris = load_iris()
X, y = iris.data, iris.target

# Split into training (80%) and testing (20%) sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.4, random_state=42)

# Train KNN classifier (k=5)
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

# Predict on test data
y_pred = knn.predict(X_test)

# Print correct and incorrect predictions
print("\n Predictions:")
for i in range(len(y_test)):
    label_pred = iris.target_names[y_pred[i]]
    label_actual = iris.target_names[y_test[i]]
    status = "Correct" if y_pred[i] == y_test[i] else " Incorrect"
    print(f"Sample {i}: Predicted = {label_pred}, Actual = {label_actual} → {status}")

# Print overall accuracy
```

```
accuracy = (y_pred == y_test).mean() * 100
print(f"\nModel Accuracy: {accuracy:.2f}%")
```

Output:

```
Predictions:
Sample 0: Predicted = versicolor, Actual = versicolor → Correct
Sample 1: Predicted = setosa, Actual = setosa → Correct
Sample 2: Predicted = virginica, Actual = virginica → Correct
Sample 3: Predicted = versicolor, Actual = versicolor → Correct
Sample 4: Predicted = versicolor, Actual = versicolor → Correct
Sample 5: Predicted = setosa, Actual = setosa → Correct
Sample 6: Predicted = versicolor, Actual = versicolor → Correct
Sample 7: Predicted = virginica, Actual = virginica → Correct
Sample 8: Predicted = versicolor, Actual = versicolor → Correct
Sample 9: Predicted = versicolor, Actual = versicolor → Correct
Sample 10: Predicted = virginica, Actual = virginica → Correct
Sample 11: Predicted = setosa, Actual = setosa → Correct
Sample 12: Predicted = setosa, Actual = setosa → Correct
Sample 13: Predicted = setosa, Actual = setosa → Correct
Sample 14: Predicted = setosa, Actual = setosa → Correct
Sample 15: Predicted = versicolor, Actual = versicolor → Correct
Sample 16: Predicted = virginica, Actual = virginica → Correct
Sample 17: Predicted = versicolor, Actual = versicolor → Correct
Sample 18: Predicted = versicolor, Actual = versicolor → Correct
Sample 19: Predicted = virginica, Actual = virginica → Correct
Sample 20: Predicted = setosa, Actual = setosa → Correct
Sample 21: Predicted = virginica, Actual = virginica → Correct
Sample 22: Predicted = setosa, Actual = setosa → Correct
Sample 23: Predicted = virginica, Actual = virginica → Correct
Sample 24: Predicted = virginica, Actual = virginica → Correct
Sample 25: Predicted = virginica, Actual = virginica → Correct
Sample 26: Predicted = virginica, Actual = virginica → Correct
Sample 27: Predicted = virginica, Actual = virginica → Correct
Sample 28: Predicted = setosa, Actual = setosa → Correct
Sample 29: Predicted = setosa, Actual = setosa → Correct
Sample 30: Predicted = setosa, Actual = setosa → Correct
Sample 31: Predicted = setosa, Actual = setosa → Correct
Sample 32: Predicted = versicolor, Actual = versicolor → Correct
Sample 33: Predicted = setosa, Actual = setosa → Correct
Sample 34: Predicted = setosa, Actual = setosa → Correct
Sample 35: Predicted = virginica, Actual = virginica → Correct
Sample 36: Predicted = versicolor, Actual = versicolor → Correct
Sample 37: Predicted = setosa, Actual = setosa → Correct
Sample 38: Predicted = setosa, Actual = setosa → Correct
Sample 39: Predicted = setosa, Actual = setosa → Correct
Sample 40: Predicted = virginica, Actual = virginica → Correct
Sample 41: Predicted = versicolor, Actual = versicolor → Correct
Sample 42: Predicted = versicolor, Actual = versicolor → Correct
Sample 43: Predicted = setosa, Actual = setosa → Correct
Sample 44: Predicted = setosa, Actual = setosa → Correct
```

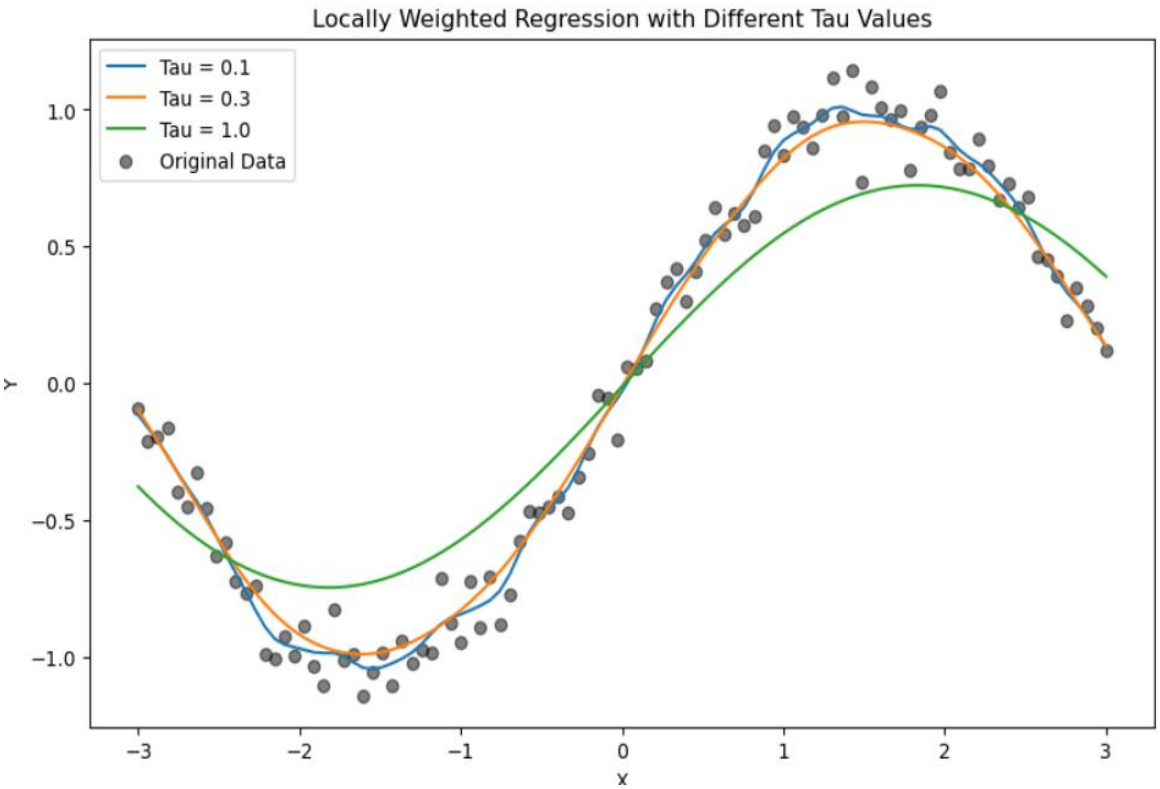
```
Sample 45: Predicted = versicolor, Actual = versicolor → Correct
Sample 46: Predicted = versicolor, Actual = virginica → Incorrect
Sample 47: Predicted = virginica, Actual = virginica → Correct
Sample 48: Predicted = versicolor, Actual = versicolor → Correct
Sample 49: Predicted = virginica, Actual = virginica → Correct
Sample 50: Predicted = versicolor, Actual = versicolor → Correct
Sample 51: Predicted = virginica, Actual = virginica → Correct
Sample 52: Predicted = versicolor, Actual = versicolor → Correct
Sample 53: Predicted = setosa, Actual = setosa → Correct
Sample 54: Predicted = virginica, Actual = virginica → Correct
Sample 55: Predicted = versicolor, Actual = versicolor → Correct
Sample 56: Predicted = setosa, Actual = setosa → Correct
Sample 57: Predicted = setosa, Actual = setosa → Correct
Sample 58: Predicted = setosa, Actual = setosa → Correct
Sample 59: Predicted = versicolor, Actual = versicolor → Correct

Model Accuracy: 98.33%
```

9. Develop a program to implement the non-parametric Locally Weighted Regression algorithm, fitting data points and visualizing results.

Code:

```
import numpy as np
import matplotlib.pyplot as plt

# Generate Synthetic Dataset
np.random.seed(42)
X = np.linspace(-3, 3, 100)  # 100 points between -3 and 3
Y = np.sin(X) + np.random.normal(0, 0.1, 100)  # Sin function with noise

# Gaussian Kernel Function
def get_weights(query_point, X, tau):
   return np.exp(-((X - query_point) ** 2) / (2 * tau ** 2))

# Locally Weighted Regression Function
def locally_weighted_regression(query_x, X, Y, tau):
   X_matrix = np.c_[np.ones(X.shape[0]), X]  # Add Bias Term
   W = np.diag(get_weights(query_x, X, tau))  # Compute weight matrix
   theta = np.linalg.pinv(X_matrix.T @ W @ X_matrix) @ X_matrix.T @ W @ Y  # Solve normal
equation
   return np.array([1, query_x]) @ theta  # Prediction for query_x

# Make Predictions for All X Points
tau_values = [0.1, 0.3, 1.0]  # Different bandwidths
plt.figure(figsize=(10, 6))

for tau in tau_values:
   Y_pred = np.array([locally_weighted_regression(x, X, Y, tau) for x in X])
   plt.plot(X, Y_pred, label=f'Tau = {tau}')
# Plot the Original Data
plt.scatter(X, Y, color='black', label="Original Data", alpha=0.5)
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Locally Weighted Regression with Different Tau Values")
plt.legend()
plt.show()
```

Output:


Locally Weighted Regression with Different Tau Values
Tau = 0.1
Tau = 0.3
Tau = 1.0
Original Data
1.0
0.5
0.0
−0.5
−1.0
Y
−3
−2
−1
0
1
2
3
x

10. Implement a Q-learning algorithm to navigate a simple grid environment, defining the reward structure and analyzing agent performance.

Algorithm:

1. Initialize Q-table with zeros
2. Define learning parameters: alpha, gamma, epsilon, episodes
3. For each episode:
   a. Set state to (0,0)
   b. While state is not (3,3):
      i. Choose action using ε-greedy policy
      ii. Move to next state
      iii. Get reward
      iv. Update Q-table using Q-learning formula
      v. Set next state as current state
   c. Print reward every 50 episodes to track progress
4. Test the trained model:
   a. Start from (0,0)
   b. Follow the best Q-values to reach (3,3)
   c. Print optimal path taken

Code:

```
import numpy as np
import random

# Grid Size and Actions (Up, Down, Left, Right)
GRID_SIZE = 4
ACTIONS = [(0, -1), (0, 1), (-1, 0), (1, 0)]  # (Up, Down, Left, Right)
GOAL = (3, 3)

# Initialize Q-table (4x4 grid with 4 actions)
Q_table = np.zeros((GRID_SIZE, GRID_SIZE, 4))
# Hyperparameters
alpha = 0.1  # Learning rate
```

```
gamma = 0.9  # Discount factor
epsilon = 0.1  # Exploration rate
episodes = 300  # Training episodes

# Function to get next state
def get_next_state(state, action):
    next_state = (state[0] + ACTIONS[action][0], state[1] + ACTIONS[action][1])
    return next_state if 0 <= next_state[0] < GRID_SIZE and 0 <= next_state[1] < GRID_SIZE else
state

# Reward Function
def get_reward(state):
    return 10 if state == GOAL else -1  # +10 for goal, -1 for each step

# Q-Learning Algorithm
for episode in range(episodes):
    state = (0, 0)  # Start position
    total_reward = 0  # Track rewards per episode

    while state != GOAL:
        # Choose action (ε-greedy)
        action = random.randint(0, 3) if random.uniform(0, 1) < epsilon else
np.argmax(Q_table[state])
        next_state = get_next_state(state, action)
        reward = get_reward(next_state)
        total_reward += reward

        # Q-value Update
        Q_table[state][action] += alpha * (reward + gamma * np.max(Q_table[next_state]) -
Q_table[state][action])
        state = next_state

    # Print progress every 50 episodes
    if (episode + 1) % 50 == 0:
        print(f"Episode {episode+1}: Total Reward = {total_reward}")

# Test the Agent's Learned Path
state = (0, 0)
path = [state]
while state != GOAL:
    action = np.argmax(Q_table[state])  # Choose best action
    state = get_next_state(state, action)
    path.append(state)

print("\nFinal Trained Q-Table:")
```

```
print(Q_table)  # Print the Q-values for all states
print("\nOptimal Path Taken by Agent:", path)  # Show learned path
```

Output:

```
Episode 50 : Total Reward = 4
Episode 100: Total Reward = 5
Episode 150: Total Reward = 5
Episode 200: Total Reward = 5
Episode 250: Total Reward = 3
Episode 300: Total Reward = 5

Final Trained Q-Table:
[[[-1.09117823  1.80979976 -0.58386923 -0.79978892]
  [-0.28074757 -1.0070673   0.06769445  3.12199994]
  [-0.78071679 -0.69198803 -0.67934652  1.16289447]
  [-0.52492694 -0.4900995  -0.4900995  -0.4390759 ]]

 [[-1.30225965  2.41935867 -1.10253905 -1.20635037]
  [-0.06657971  0.89701519 -0.07603423  4.57999999]
  [-0.30649356 -0.360271   -0.3160428   4.87053743]
  [-0.199      -0.29539    -0.199       0.264581  ]]

 [[-0.67715374  1.86888611 -0.79471794 -0.66909702]
  [-0.44908084  2.7845103   1.32323824  6.2       ]
  [-0.215929   -0.1        -0.01429821  7.82985999]
  [-0.1        -0.1        -0.109       4.0951    ]]

 [[-0.48136681  4.10827514 -0.42360383 -0.5621488 ]
  [ 0.75880108  8.          1.75751839  3.65650663]
  [ 2.834704   10.          1.54871535  1.43768997]
  [ 0.          0.          0.          0.        ]]]

Optimal Path Taken by Agent: [(0, 0), (0, 1), (1, 1), (2, 1), (3, 1), (3, 2),
(3, 3)]
```